Response to JSS Editor's Preliminary Review

BayesianDEB: A Bayesian Framework for Dynamic Energy Budget Modelling in R

Branimir K. Hackenberger
on behalf of the authors

2026-05-11

We thank the editor for the close reading of *BayesianDEB: A Bayesian Framework for Dynamic Energy Budget Modelling in R*. The comments group naturally into three areas: the R class system, the replication material, and the package vignettes. We have worked through all of them. Each comment is quoted below, word for word or as a faithful summary, followed by our reply and the commits and files it touches in version **0.2.0** of the package.

The work lives on the `jss-revision` branch of https://github.com/sciom/BayesianDEB and is tagged `v0.2.0`. The final package is `BayesianDEB_0.2.0.tar.gz` and the replication archive is `BayesianDEB_replication.zip`.

1 Class system

1.1 `bdeb_data` and `bdeb_model` are missing `summary()` and `plot()`

> "`methods(class = "bdeb_data")` shows only `print`. Journal of Statistical Software requires that at least `summary` and `plot` are also implemented for every class in the package. The same remark holds for `bdeb_model`."

Response. Added. `bdeb_data`, `bdeb_model` and `bdeb_prior` now carry a complete `print()` / `summary()` / `plot()` trio. Each `summary()` method returns a tidy `summary.bdeb_data` / `summary.bdeb_model` / `summary.bdeb_prior` object with its own `print()` method, and each `plot()` method gives an overview of the data, the model structure, or the prior densities.

Reference. Tasks B5/B6/B7 of the revision plan; commits in `R/data_prep.R`, `R/model_spec.R`, `R/priors.R`. Tests added in `tests/testthat/test-api-contracts.R`.

1.2 `bdeb_diagnose()` returns a plain `list`

> "`bdeb_diagnose` prints a very long information and returns an object of class `list`. This should be improved. It would be better that `bdeb_diagnose` returns a specific class object with methods to inspect it and that its `print` is the output of the `print` method for this class."

Response. `bdeb_diagnose()` now returns a `bdeb_diagnostics` S3 object with `print()`, `summary()` and `plot()` methods. It no longer prints to the console as a side effect: the output comes from `print(d)`, or from autoprinting at the top level. The `plot()` method takes `type = "rhat"` and `type = "ess"`, and the old list-style access (`d$n_divergent`, `d$summary`) still works for backward compatibility.

Reference. Task B5; `R/diagnostics.R`.

1.3 `predict(fit)` returns an unstructured long list

> "`predict(fit)` is a long list with multiple entries. We would expect a specific class for this object as well, to improve readability of the output for users and allow the implementation of methods that would allow inspecting this object."

Response. `bdeb_predict()`, the function behind `predict.bdeb_fit`, now returns a `bdeb_prediction` S3 object with `print()`, `summary()` and `plot()` methods. `print()` reports the model type, the prediction horizon and the posterior summary widths; `summary()` hands back a tidy data frame with `time` / `lower` / `median` / `upper` columns.

Reference. Task B6; `R/fit.R`, `R/plot.R`.

1.4 `bdeb_fit` has too few methods

> "`methods(class = "bdeb_fit")` returns `[coef plot predict print summary]` which seems to be a bit limited. For model classes, we would expect more methods allowing to inspect the output and assess the quality of the results (see, e.g., `methods(class = "lm")`), including `confint`, `residuals`, ..."

Response. We added the `lm`-style methods that make sense for a Bayesian DEB fit:

- `confint()`: posterior credible intervals.
- `fitted()`: posterior median/mean of $\hat{L}_i$.
- `residuals()`: observed minus fitted.
- `nobs()`: observation count.
- `vcov()`: posterior covariance of model parameters.
- `logLik()`: log-pointwise predictive density (`lppd`).

`fitted()`, `residuals()` and `logLik()` apply only to the `"individual"` and `"growth_repro"` model types; on `"hierarchical"` and `"debtox"` fits, where a single-observation residual has no clear meaning, they stop with an explanatory error.

Reference. Tasks B10/B11; `R/fit.R`.

1.5 `bdeb_derived()` should be a method on `bdeb_fit`

> "`bdeb_derived` starts with `if (!inherits(fit, "bdeb_fit"))` which shows that it could have been implemented as a method rather than a simple function to clarify the class structure for the user."

Response. `bdeb_derived()` is now an S3 generic, with `bdeb_derived.bdeb_fit` as its method. The call users write is unchanged, but dispatching this way makes the class structure visible and leaves room for methods on prior-only and simulated objects later.

Reference. Task B7; `R/fit.R`.

1.6 `summary(fit)` and `bdeb_summary(fit)` are redundant

> "`summary(fit)` and `bdeb_summary(fit)` seem to be the same thing, which questions the reason why the second has been implemented."

Response. `summary.bdeb_fit()` is now the main way to summarise the posterior: it takes `pars` and `prob` arguments and returns a `posterior::draws_summary` data frame, in the spirit of `summary.lm()`. `bdeb_summary()` is deprecated. The wrapper still runs, but it raises a lifecycle deprecation warning on first use and will go in a future release.

Reference. Task B9; `R/fit.R`, `R/diagnostics.R`.

1.7 Input validation is incomplete

> "Functions do not always check their inputs. All inputs must be checked to assess that their value is what is expected and returns comprehensive messages to users when this is not the case."

Response. Every exported function now checks its inputs through one shared set of internal assertions (`assert_positive`, `assert_finite_scalar`, `assert_probability`, and so on). Error messages go through `cli::cli_abort()` with named bullets and suggested fixes. Input validation is now covered by more than 180 contracts; `tests/testthat/test-input-validation.R` (63 `expect_error` checks) and `test-api-contracts.R` (38 checks) between them exercise the entry points users hit most.

Reference. Task B8; `R/utils.R` for the assertions, individual R files for their use.

2 Replication material

2.1 Replication material organisation and README

> "A single zip folder should be submitted which contains all scripts and results. If more than one script needs to be provided, a README should be added which explains which part of the replication material reproduces which part of the manuscript."

Response. The new `replication/` directory ships as a single zip (`BayesianDEB_replication.zip`), laid out like this:

```
replication/
  README.md                # one-page guide: what each script does
  00_setup.R               # libraries, palette, BDEB_MODE
  01_illustrations.R       # Section 5 of the manuscript
  02_validation.R          # Section 6
  03_case_studies.R        # Section 7
  comparison_debinfer.R    # Section 9 auxiliary comparison
  data/                    # bundled inputs (curves.txt, ...)
  outputs/                 # generated figures and tables
  sbc/                     # long-running SBC scripts
```

`README.md` maps each script to the manuscript sections it produces, gives the run time in both `lite` and `full` modes, and lists the required and optional packages.

Reference. Phase 3 (tasks A1–A7); `replication/README.md`.

2.2 Manuscript code missing from replication

> "All code shown in the manuscript should also be included in the replication scripts. E.g., page 7 contains `prior_species("Eisenia_fetida")`. However, we were unable to find this code in the replication scripts provided."

Response. Every code listing in the manuscript is now run in the replication scripts. `prior_species("Eisenia_fetida")` is called in `replication/01_illustrations.R` just before the matching `bdeb_fit()` call from Section 5.1, and the `obs_student_t()` one-liner from p. 18 is now constructed in Section 5.1 as well, so the audit is complete, even though nothing is fitted with it there.

Reference. `replication/01_illustrations.R` lines for `prior_species()` and `obs_student_t()`.

2.3 Differences between replication output and manuscript

> "Running replication material shows that there are slight differences between the outputs produced by the replication material and what is shown in the manuscript. Please fix this or explain why identical results cannot be obtained by specifying the specific environment that allowed to obtain those results in the manuscript."

Response. Those differences came from two things in the earlier scripts: two figures used an unfixed seed, and the exact Stan, cmdstanr and R versions behind the manuscript were not recorded. We have:

- fixed a global seed (`set.seed(20260418)`) in `00_setup.R` and pass it to every `bdeb_fit()` and `plot_*()` call through explicit `seed =` arguments;
- recorded the rendering environment in `replication/outputs/sessionInfo.txt`, captured with `BayesianDEB::bdeb_session_info()` and `utils::sessionInfo()`;
- with this environment, `BDEB_MODE=full Rscript ...` reproduces every manuscript figure and table bit for bit.

A small residual difference ($\sim 10^{-4}$ in the posterior summaries) still shows up across platforms, because Stan's adaptive HMC depends on the order of floating-point reductions in `reduce_sum`; we note this in `replication/README.md`.

Reference. Tasks A3/A4; `replication/00_setup.R`, `replication/outputs/sessionInfo.txt`.

2.4 CVode mxstep informational messages

> "The replication material repeatedly produces these warnings: `CVode(... CV_NORMAL) failed with error flag -1: The solver took mxstep internal steps but could not reach tout.` That seems to be important to fix or at least provide an explanation and guide users in how to interpret these warnings."

Response. Stan's stiff BDF ODE solver prints these during HMC warm-up, when the sampler wanders into parameter combinations that do not fit the data. They are *informational* rather than errors, and Stan documents them as such: the integrator hands control back, the leapfrog step is rejected by the Metropolis criterion, and the chain carries on.

We made two changes:

1. Increased `ode_bdf_tol max_num_steps` from `1e4` to `1e5` in all four Stan models (`inst/stan/bdeb_*.stan`). This removes roughly two-thirds of the messages on the default 300-iteration warm-up.
2. Added a paragraph to `replication/README.md` and to the *Getting started* vignette ("ODE solver messages") that quotes the Stan reference manual and explains when the messages point to a real problem (they recur past warm-up, alongside divergent transitions) and when they are just harmless warm-up exploration (sporadic, warm-up only).

Reference. Task A4 / C1; `inst/stan/bdeb_*.stan`, `replication/README.md`, `vignettes/getting_started.Rmd`.

2.5 External `curves.txt` dependency

> "The replication material contains `curves <- read.delim( "https://raw.githubusercontent.com/JeromeMathieuEcology/EGrowth/master/curves.txt")`. This file should also be directly included in the replication material to reduce the dependency on availability from external unreliable sources."

Response. `curves.txt` now travels with the material at `replication/data/curves.txt` (SHA-256 `19e131b8...`, recorded in `replication/data/CHECKSUMS.txt`). The scripts read the local copy, and the URL-based form is gone. Its provenance and licence (CC-BY-4.0, Mathieu 2024) are noted in `replication/README.md`.

Reference. Task A5; `replication/data/curves.txt`, `replication/data/CHECKSUMS.txt`.

2.6 `eisenia_real_data.R` fails with `T = 298.15`

> "Running `eisenia_real_data.R` fails with `Error in bdeb_model(): Temperature list missing: T_obs.`"

Response. In version 0.1.4 we renamed the temperature-list field `T` to `T_obs`, both to match the Stan-side name and to stop it shadowing R's built-in `T = TRUE`. The replication scripts now follow suit: every `temperature = list(...)` call uses `T_obs = ...`. We weighed a backward-compatibility shim but left it out to keep the v0.2.0 API clean.

Reference. Task A6; `replication/03_case_studies.R`.

2.7 Computing resource budget

> "We would expect that the replication material runs within a reasonable amount of time using standard computing resources. Excessive use of computing resources should be avoided when providing examples to demonstrate the use of the package."

Response. Each replication script runs in one of two modes, chosen with the `BDEB_MODE` environment variable:

- `lite` (default): `chains = 2`, `iter_warmup = 300`, `iter_sampling = 300`. Reproduces the qualitative shape of every figure and table. Total wall-clock on a 4-core laptop: $<$ **30 minutes** for `01` + `02` + `03`.
- `full` (manuscript-equivalent): `chains = 4`, `iter_warmup = 1000`, `iter_sampling = 1000`. Reproduces bit-identical posteriors and diagnostics. Total wall-clock on a 4-core laptop: ≈ 3 **hours**.

`README.md` documents the choice. The SBC calibration scripts (`replication/sbc/`) stay in their own sub-folder: they are long-running by nature (~ 10 h) and are not needed to reproduce the manuscript figures, so their results ship as cached `.rds` files.

Reference. Task A2; `replication/00_setup.R` (mode dispatch), `replication/README.md`.

3 Vignettes and examples

3.1 `bdeb_model` has no examples

> "`example("bdeb_model")` warns `'bdeb_model' has a help file but no examples`. Since `bdeb_model` seems to be one of the main functions of the package, it is important that its help page is accompanied with an example."

Response. We added runnable examples to every main function whose example does not need CmdStan: `bdeb_data`, `bdeb_model`, `bdeb_tox`, `bdeb_prior_predictive`, `prior_species`, and all the `prior_*` constructors. Each runs on bundled datasets in well under a second under R CMD check defaults, which we confirmed with `R CMD check --as-cran --run-donttest`.

Reference. Task C2; `R/*.R` (Roxygen `@examples` blocks).

3.2 `bdeb_fit` examples in `\dontrun{}`

> "`example(bdeb_fit)` shows that all code is in `dontrun`. It would seem preferable to conditionally run the code after checking that the external CmdStan toolchain is available, similar to what is suggested for suggested packages in Writing R Extensions."

Response. `bdeb_fit()` keeps its `\dontrun{}` example for two reasons: compiling a Stan model and then running MCMC goes well past the 5-second-per-example limit in R CMD check, and the CmdStan toolchain has to be installed and tested deliberately rather than skipped on failure. The Roxygen block now says this in a comment that shows up in the rendered Rd, so a reader can see why the example is wrapped.

For coverage we lean on two full vignettes (*Getting started*; *Case study: Eisenia / Folsomia*) and the replication scripts, all of which guard their Stan chunks with `requireNamespace("cmdstanr", quietly = TRUE)`.

Reference. Task C2 (Roxygen comment); Task C1 (vignette conditionality, see 3.3).

3.3 Vignette code commented out

> "All code in vignettes seems to be commented out, see for example https://cran.r-project.org/web/packages/BayesianDEB/vignettes/case_study_eisenia_folsomia.R. It would seem preferable to check for availability of cmdstanr and then conditionally allow to run the code."

Response. Both vignettes now gate every Stan-touching chunk on `requireNamespace("cmdstanr", quietly = TRUE)`, following the *Writing R Extensions* advice for suggested packages: where cmdstanr is installed *and* `cmdstanr::cmdstan_version()` returns a non-`NULL` version string, the chunk runs and shows the posterior; otherwise it is skipped and a message explains how to install CmdStan.

To keep the rendered HTML small, each fit drops to `lite` mode (`chains = 2, iter = 300+300`) when the build happens on CRAN; on a full installation the same code still draws every figure from the article.

Reference. Task C1; `vignettes/getting_started.Rmd`, `vignettes/case_study_eisenia_folsomia.Rmd`.

3.4 `plot(fit, type = "pairs")` returns NULL

> "Running the code it would seem that no plot is created with: `plot(fit, type = "pairs", pars = c("p_Am", "p_M", "kappa"))` `NULL`"

Response. The old `plot.bdeb_fit(type = "pairs")` called `bayesplot::mcmc_pairs()`, which returns a `bayesplot_grid` (a `gtable`), and then tried `result + ggplot2::labs()`, which is undefined for `gtable` objects and quietly returned `NULL`.

The method now returns the `bayesplot_grid` directly, and the requested parameters reach `bayesplot::mcmc_pairs()` through `posterior::subset_draws()`, so the `pars` argument is honoured exactly. Tests in `tests/testthat/test-plot.R` check that the return value is non-NULL and that the grid has the right dimensions.

Reference. Task C1; `R/plot.R`.

4 Additional improvements (not requested, but bundled)

Since the class-system rework reached into most of the package, we used the occasion to:

- Add LOO and WAIC predictive-density comparison (`bdeb_loo()`).
- Add the `obs_student_t()` observation family for robust growth fits.
- Add Arrhenius temperature correction to all four Stan models (previously only `"individual"`).
- Add within-chain parallelism via `reduce_sum` to the `"hierarchical"` and `"debtox"` models, with the `threads_per_chain` argument exposed in `bdeb_fit()`.
- Grow the test suite from 58 `test_that` blocks in version 0.1.0 to 395 blocks across 18 files (API contracts, scientific consistency, snapshot regression, deep validation, end-to-end integration, input validation, class methods), with over 180 input-validation contracts as detailed in 1.7.

All of these are recorded in `NEWS.md` under the 0.2.0 entry.

5 Submission summary

The revised submission consists of:

- `BayesianDEB_0.2.0.tar.gz`: the package, passes `R CMD check --as-cran` with **0 errors / 0 warnings / 1 note** (the single NOTE is the informational "unable to verify current time", produced when the host has no network access to the time server; it is unrelated to package contents).
- `BayesianDEB_replication.zip`: replication material, with `README.md`, bundled `curves.txt`, and `lite` / `full` modes.
- The present document (`response_to_editor_round1.tex`).

We thank the editor and reviewers for the thorough, constructive feedback, which has improved the package considerably.

Branimir K. Hackenberger
on behalf of the authors
2026-05-11